Blockchain Protocol Security Analysis Report

Customer: DevolvedAI

Date: 23/08/2024

We express our gratitude to the DevolvedAI team for the collaborative engagement that enabled the execution of this Blockchain Protocol Security Assessment.

ArgoChain-SDK is specifically designed for developers eager to explore and innovate within the ArgoChain ecosystem.

Document

Name	Blockchain Protocol Review and Security Analysis Report for DevolvedAI
Approved By	Luciano Ciattaglia
Website	https://devolvedai.com
Changelog	28/06/2024 - Preliminary Report
	12/07/2024 - AMM + Bridge
	23/08/2024 - Final Report
Platform	Polkadot
Language	Rust, Solidity
Tags	Layer 1, Substrate, Fork, AMM, Bridge
Methodology	https://hackenio.cc/blockchain_methodology

Review Scope

Repository	https://github.com/Devolved-AI/Argochain
Commit	19e66003f426e6aba0e5f5bf26afb578450dc4af

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

11	7	3	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	7
Medium	1
Low	3

Vulnerability	Severity
F-2024-4277 - Unexpected Swap Rate due to Lack of Slippage Check	High
F-2024-4281 - Centralization Threat and Reward Imbalance Due to Era Payout Mechanism	High
F-2024-4290 - Pool Liquidity Loss due to Insecure Pools Management	High
F-2024-4295 - Deposited Liquidity Misallocation due to Lack of Cashback	High
F-2024-4297 - Locked Tokens Are Transferred To Arbitrary Address	High
F-2024-4303 - Vulnerable Dependencies and Outdated Substrate Version	High
F-2024-4836 - Reserves to Balances Inconsistency due to Incorrect Fee Withdrawal Mechanism	High
F-2024-4282 - Inadequate Calculation of Residual Reward Amount	Medium
F-2024-4151 - Lack Of Two-Step Ownership Transfer Pattern	Low
F-2024-4153 - Return Value Of transferFrom Not Checked	Low
F-2024-4298 - Lack Of Two-Step Ownership Transfer Pattern	Low

Commit History and Changes

The following is a detailed history of the modifications made to the original Substrate codebase by the Argochain project, based on specific commits in their GitHub repository. Each commit is categorized into those with real code changes and those focused on naming or rebranding. The analysis reveals that the majority of the commits were centered around rebranding Substrate rather than introducing significant new functionality or security changes at the initial stage of the audit. However, the DevolvedAI team improved the technical ground of the project significantly during the next stages, reaching a new, more mature, and technically developed protocol.

Commits with Code Changes

Commit [2b78446b50848d53afe6c5abc583fe2570663102]

- **Summary**: Configuration updates.
- **Details**: Added details for `tokenSymbol` ("AGC"), `tokenDecimals` (18), and `ss58Format` (33) to multiple configuration files. These additions were made in the files `customSpec.json`, `customSpecRaw.json`, and `chain_spec.rs`, among others. The changes help define the token's attributes within the blockchain's configuration.

Commit [f08e98e190928e700f205ec8ed70d7b466bbc258]

- **Summary**: Updates including genesis block and EVM Compatibility.
- **Details**: Adds several new configurations and parameters for EVM and Ethereum compatibility (outside of scope). Key changes include the addition of `EVMConfig` and `EthereumConfig` for handling EVM accounts, balances, and storage, along with `dynamic_fee` and `base_fee` parameters. The commit also includes various other updates to the runtime genesis configuration, like the inclusion of default configurations for assets, nomination pools, and transaction settings, enhancing the blockchain's initial setup and compatibility.

Commit [9c48b61b2b4957b384c00b8f0ce0665c0b989176]

- **Summary**: Addition of EVM support.
- **Details**: Added support for the Ethereum Virtual Machine (EVM), integrating EVM functionality into the Argochain network. This change is outside the current audit scope.

Commit [100f150d025e6d314b9a1f4d9d049ecdccd249cf]

- **Summary**: Updated chain specifications.
- **Details**: Changes to the chain specification parameters.

Commit [5a0543569ec136e6e9cf455e1add80f9e39cff01]

- **Summary**: Change to constant value.
- **Details**: Changed `u64 = 997` to `u64 = 1299`. This change affects a specific constant value in the code, modifying an operational parameter or limit.

Commit [47b59ee2bea435b99fa6f2478af90f2060e8aa85]

- **Summary**: Changes to `chain_spec.rs`.

- **Details**: Changes to three hardcoded Ethereum addresses used in the testnet genesis configuration. Each of the three addresses, which represent EVM accounts with pre-set balances, was updated to new addresses, ensuring that the genesis block configuration aligns with the latest project requirements.

Commit [dc575d4db21ee3d2496144c88770224cb608128f]

- **Summary**: Adjustments to network parameters.
- **Details**: This commit made specific changes to the network parameters and configurations, ensuring the network's operational settings align with Argochain's requirements. Including genesis configuration parameters and adjustments to the runtime configuration, such as modifying values for the EVM accounts and balances, updating default parameters for system modules, and ensuring compatibility with the latest project specifications.

Commit [baec60df86d4e0f03c6be37fdfe9fd78aed076bf]

- **Summary**: Tokenomics and runtime configurations.
- **Details**: Introduces several updates primarily focused on the tokenomics and runtime configurations. Key changes include modifications to the runtime/src/constants.rs and runtime/src/lib.rs files, adjustments to staking parameters, and the implementation of a custom era payout structure for staking rewards.

Commits with Rebranding or No Relevant Changes

Commit [7bd237880646a8107f05e95c0a4c0c504e99d1a5]

- **Summary**: Configuration changes and renaming.
- **Details**: Renaming of project identifiers from Substrate to Argochain.

Commit [848b632c555d317b0357e461755f38432d6c9811]

- **Summary**: Token name change.
- **Details**: Modified the token name to reflect Argochain’s native token, AGC.

Commit [4836b373a3d1733c909d7c1ab88ec9ac50854123]

- **Summary**: Configuration file changes.
- **Details**: Changed configuration files without altering the genesis state.

Commit [a80557235c078cf9d2312e3d2b831523e9851773]

- **Summary**: Added `rust-setup.md` file.
- **Details**: Added a Markdown file with setup instructions for Rust, without changing code functionality.

Commit [2409d284ead47a2c0fc16779655d458c81380dc7]

- **Summary**: Removed comments.
- **Details**: Removed comments from the code without changing functionality.

Commit [275698a19d07f702c2cad0b8bdf25e1811daf0da]

- **Summary**: Added Markdown files.

- **Details**: Added 3 Markdown files for documentation purposes.

Commit [85d4af8ce67ece08b29e7776e85742f0b501ccf0]

- **Summary**: Minor formatting change.
- **Details**: Added a space in the code without making functional changes.

Commit [3f71cda06d4b7e2f9e7d4ddfcc502121b0e23fe5]

- **Summary**: Addition of MD files.
- **Details**: Added Markdown documentation files without changing code functionality.

Commit [92387003bcdc4c6f3796d29956469219006dc986]

- **Summary**: Edited README.
- **Details**: Updated the README file with new project information and branding.

Commit [b2d243517b034e9a3532fbf406bc0d2075abbfe6]

- **Summary**: Deleted a file.
- **Details**: Removed a file from the codebase without impacting functionality.

Commit [d147dc9b107dcd88e589bb4c8890349a09f73dcd]

- **Summary**: Added bootnodes.
- **Details**: Added bootnodes to the network configuration.

Commit [19e66003f426e6aba0e5f5bf26afb578450dc4af]

- **Summary**: Documentation rebranding.
- **Details**: Updated rebranding in the project's documentation and guides.

Commits Related to Project Setup

Commit [542ad5e359c2795e15655c240cff4579f4ce712f]

- **Summary**: Initial upload of Substrate code.
- **Details**: This commit represents the initial upload of the Substrate framework into the Argochain repository. It includes all the core components and modules of Substrate without any modifications.

Commit [51ff126c9fea843c8a492dbdde45d7fea1724147]

- **Summary**: Added Frontier.
- **Details**: Added code from https://github.com/polkadot-evm/frontier.

Commit [0a3718c719e240e61b2f3440722e1151754524f8]

- **Summary**: Added dependencies.
- **Details**: Added dependencies originally from Substrate to the project.

Total Commits and Focus Analysis

- **Total Commits Analyzed**: 23
- **Commits with Code Changes**: 8

- **Commits with Rebranding or No Relevant Changes**: 12
- **Commits Related to Project Setup**: 3

The focus of the majority of the commits in the Argochain repository was on rebranding the Substrate framework rather than introducing significant new functionality or substantial code modifications at the beginning of the audit. This rebranding effort includes changes to project identifiers, metadata, and documentation. As a result, no significant security issues were identified during the first stage of the audit, as the core code modifications over Substrate that could potentially introduce vulnerabilities were minimal. During supplementary reviews of new components added to the scope, security issues were raised, which are detailed in the Findings section of this audit report; most of the issues were promptly addressed by the DevolvedAI team.

Table of Contents

System Overview **10**

System Architecture 10

Risks **12**

Findings **13**

Vulnerability Details 13

F-2024-4277 - Unexpected Swap Rate Due To Lack Of Slippage Check - High 13

F-2024-4281 - Centralization Threat And Reward Imbalance Due To Era Payout Mechanism - High 16

F-2024-4290 - Pool Liquidity Loss Due To Insecure Pools Management - High 19

F-2024-4295 - Deposited Liquidity Misallocation Due To Lack Of Cashback - High 22

F-2024-4297 - Locked Tokens Are Transferred To Arbitrary Address - High 25

F-2024-4303 - Vulnerable Dependencies And Outdated Substrate Version - High 27

F-2024-4836 - Reserves To Balances Inconsistency Due To Incorrect Fee Withdrawal Mechanism - High 30

F-2024-4282 - Inadequate Calculation Of Residual Reward Amount - Medium 33

F-2024-4151 - Lack Of Two-Step Ownership Transfer Pattern - Low 35

F-2024-4153 - Return Value Of TransferFrom Not Checked - Low 36

F-2024-4298 - Lack Of Two-Step Ownership Transfer Pattern - Low 37

Observation Details 38

F-2024-4152 - Contract Has No Authorised Functions - Info 38

F-2024-4275 - Revert Messages Optimization - Info 39

F-2024-4276 - Floating Pragma - Info 40

F-2024-4293 - Lack Of Token Standard Compliance - Info 41

F-2024-4294 - Public Functions That Should Be External - Info 42

F-2024-4296 - Reentrancy Guard Duplication - Info 43

F-2024-4304 - Presence Of Unused Template Pallet - Info 44

F-2024-4818 - Unnecessary Access Restriction On A View Function - Info 45

F-2024-4830 - Sign Of Unfinalized Code: Unused Internal Functions - Info 46

F-2024-4831 - Lack Of ERC-20 Operation Success Validation - Info 47

F-2024-4832 - Sign Of Unfinalized Code: Unused Allowance Functionality - Info 48

F-2024-4833 - Redundant View Getters For Public State Variables - Info 49

F-2024-4834 - Block WETH Functionality Due To Lock Of Native Token Receiving - Info 50

F-2024-4835 - Sign Of Unfinalized Code: Redundant Constants - Info 51

Disclaimers 52

Hacken Disclaimer 52

Technical Disclaimer 52

Appendix 1. Severity Definitions **53**

System Overview

Argochain is a blockchain protocol developed by Devolved AI. It aims to present an innovative approach to blockchain technology. The protocol has been developed by forking the Substrate framework with minimal modifications at the point of this audit (the development is still in progress and the maturity of the project is growing fast). This overview will provide a comprehensive analysis of the Argochain system while objectively highlighting the context of its development.

System Architecture

Core Components

- **Consensus Mechanism**: Argochain employs a consensus mechanism based on Substrate's default BABE (Blind Assignment for Blockchain Extension) and GRANDPA (GHOST-based Recursive Ancestor Deriving Prefix Agreement) protocols. These consensus algorithms are well-established in the Substrate ecosystem, providing robust and secure block production and finalization.
- **Runtime**: The runtime of Argochain is written in Rust, leveraging Substrate's modular runtime framework. The runtime consists of various pallets (modules) that define the core functionality of the blockchain, such as balances, governance, staking, and more. Minimal modifications have been made to the runtime, primarily involving parameter adjustments and configuration changes.
- **Networking**: Argochain's networking layer is built on top of Substrate's libp2p networking stack. This layer handles peer discovery, message propagation, and network security. The network configuration and parameters have been slightly altered to fit Argochain's specific requirements.
- **Extrinsics and Transactions**: The handling of extrinsics (transactions) in Argochain follows Substrate's default mechanism. Transactions are validated, processed, and included in blocks using the same framework provided by Substrate, with minor tweaks to transaction fees and limits.

Customizations and Enhancements

While Argochain is fundamentally based on Substrate, a few customizations have been implemented to distinguish it from a vanilla Substrate deployment:

- **Token Economics**: The native token of Argochain, AGC, has specific economic parameters, such as issuance rate, staking rewards, and transaction fees. These parameters have been fine-tuned to align with the project's goals and community expectations.
- **Governance Model**: Argochain utilizes a governance model derived from Substrate's democracy and council pallets. The configuration of these pallets has been adjusted to suit the project's governance structure, enabling community-driven decision-making.

- **Chain Specifications**: The chain specifications, including genesis state and bootnodes, have been customized to initialize the Argochain network. These changes ensure a unique network identity and facilitate the network's launch and operation.

Risks

- **Scope Definition and Security Guarantees**: The audit does not cover all code in the repository. Components outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of protocols.
- **Single Points of Failure and Control**: The project may be partially centralized, introducing single points of failure and control, due to the Era Payout Mechanism (detailed in the Findings section)
- **Absence of a Token Burn Mechanism:** The bridge contract attempts to lock and burn tokens, however, it does transfer of tokens to arbitrary address. Thus, it affects management of the tokens supply, its stability and inflation control.
- **Code Changes During the Remediation Phase:** During the remediation phase, some of the changes made to address the previously identified vulnerabilities have introduced new security risks, specifically within the AMM Contracts. While Hacken's team did review these changes separately after the audit timeline, we recommend a re-audit of the AMM code to ensure that any potential new vulnerabilities are thoroughly examined and addressed.

Findings

Vulnerability Details

F-2024-4277 - Unexpected Swap Rate due to Lack of Slippage Check - High

Description:

The contract offers token swaps based on the reserves rate, introducing the possibility of price manipulation via reserve changes (high amount swaps).

The `swap` function of the `AMM` contract lacks a minimal amount out parameter or any other slippage mechanism. This allows the contract to process the swap at any rate, regardless of significant fluctuations in reserve sizes.

```
function swap(pair, token, amountIn) ... returns (amountOut) {
...
uint256 amountIn = (amountIn * 997) / 1000;
amountOut = (reserveOut * amountInWithFee) / (reserveIn + amountInW
ithFee);

require(amountOut > 0 && amountOut <= reserveOut);

tokenIn.safeTransferFrom(msg.sender, address(this), _amountIn);
tokenOut.safeTransfer(msg.sender, amountOut);
...
}
```

This may lead to any `swap` function invocation being wrapped with transactions of increasing and decreasing the exchange rate causing the swap being performed at an unexpectedly low rate.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Classification

Impact: 4/5

Likelihood: 4/5

Severity: High

Recommendations

Remediation: Implement the minimal amount out parameter and validate it in the `swap` function to prevent swaps that deviate beyond a specified threshold from the expected rate.

Resolution: The Finding is fixed.

The `_minAmountOut` parameter validation is implemented to the `swap` function.

Evidences

Proof of Concept

Reproduce:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.22;

import "forge-std/Test.sol";
import "../src/AMM.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MockERC20 is ERC20 {
constructor(string memory name, string memory symbol) ERC20(name, s
ymbol) {}

function mint(address to, uint256 amount) public {
_mint(to, amount);
}
}

contract AMMTest is Test {
AMM public CONTRACT;
MockERC20 public T0;
MockERC20 public T1;
uint256 public ID;

address constant internal ATTACKER = address(0x12);
address constant internal LIQUIDITY_PROVIDER = address(0x13);
address constant internal USER = address(0x14);

function setUp() public {
T0 = new MockERC20("Token0", "TK0");
T1 = new MockERC20("Token1", "TK1");
CONTRACT = new AMM();
ID = CONTRACT.createPair(address(T0), address(T1));
}

function testSandwichAttack() public {
uint256 initialLiquidity = 100_000 * 10**18;
uint256 attackerPool = 10_000_000 * 10**18;
uint256 userSwap = 1_000 * 10**18;

vm.startPrank(LIQUIDITY_PROVIDER);
T0.mint(LIQUIDITY_PROVIDER, initialLiquidity);
T1.mint(LIQUIDITY_PROVIDER, initialLiquidity);

T0.approve(address(CONTRACT), initialLiquidity);
T1.approve(address(CONTRACT), initialLiquidity);

CONTRACT.addLiquidity(ID, initialLiquidity, initialLiquidity);
vm.stopPrank();

vm.startPrank(ATTACKER);
T0.mint(ATTACKER, attackerPool);
T0.approve(address(CONTRACT), attackerPool);
T1.approve(address(CONTRACT), type(uint256).max);
vm.stopPrank();
```

```
vm.startPrank(USER);
T0.mint(USER, userSwap);
T0.approve(address(CONTRACT), userSwap);
vm.stopPrank();

vm.startPrank(ATTACKER);
CONTRACT.swap(ID, address(T0), attackerPool);
vm.stopPrank();

vm.startPrank(USER);
CONTRACT.swap(ID, add
```

See more

Results:

```
[FAIL. Reason: assertion failed: 1000040316342855164715534 6 > 10000
000000000000000000000] testSandwichAttack()
```

F-2024-4281 - Centralization Threat and Reward Imbalance Due to Era Payout Mechanism - High

Description:

The issue arises from the implementation of `era_payout`, which defines the reward distribution logic for each epoch.

Argochain/runtime/src/lib.rs:1196

```
impl pallet_staking::EraPayout<Balance> for CustomEraPayout {
fn era_payout(_total_staked: Balance, _total_issuance: Balance, _er
a_duration: u64) -> (Balance, Balance) {
let total_reward = FIXED_REWARD;

let validator_reward = (total_reward * 45) / 100;
let pov_reward = (total_reward * 55) / 100;

let treasury_reward_from_validator = (validator_reward * 1) / 100;
let treasury_reward_from_pov = (pov_reward * 1) / 100;

let final_validator_reward = validator_reward - treasury_reward_fro
m_validator;
let final_pov_reward = pov_reward - treasury_reward_from_pov;

Treasury::on_unbalanced(NegativeImbalance::new(treasury_reward_from
_validator + treasury_reward_from_pov));
Balances::deposit_creating(&PovAccount::get(), final_pov_reward);

(final_validator_reward, total_reward)
}
}
```

The current logic allocates a fixed reward amount for each epoch, with almost 55% of the reward being deposited into a constant custom account defined as:

Argochain/runtime/src/lib.rs:101

```
pub const PovAccount: AccountId32 = AccountId32::new(hex!("e483f6d0
d4a9f04510d7506227a149579fd63b8c8b828d9d0b306c48aad99c67"));
```

This approach presents several risks:

Unfair Distribution: Validators receive only 45% of the total reward, while `PovAccount` receives the majority. This can be perceived as unfair by validators and might erode trust in the system.

Centralization Risk: The `PovAccount` accumulates larger rewards each epoch, which can subsequently be utilized as a stake. Over time, this accumulation can lead to a disproportionate increase in its influence compared to other validators. As a result, this presents centralization risks, with the `PovAccount` potentially exerting undue dominance over the validator ecosystem and undermining the network's decentralized integrity.

Addressing these issues is crucial to maintaining a fair and decentralized network.

Assets:

- Consensus [https://github.com/devolved-ai/argochain]

Status: Accepted

Classification

Impact: 3/5

Likelihood: 5/5

Severity: High

Recommendations

Remediation: To address the security and fairness issues in the current `era_payout` implementation, it is recommended to adjust the reward distribution to be more equitable between validators and the constant account (`PovAccount`). Instead of the current allocation of 45% to validators and 55% to the `PovAccount`, a more balanced approach would be to allocate 90% of the total reward to validators and 10% to the `PovAccount`. This adjustment ensures that validators receive fair compensation for their work, which helps to maintain trust in the system and reduces the risk of perceived unfairness.

Additionally, as in the current implementation, a small percentage (1%) of both the validator and `PovAccount` rewards should be allocated to the treasury to support the ecosystem's sustainability.

This new distribution method also mitigates the centralization risk by preventing the `PovAccount` from disproportionately increasing its stake. By promoting a more decentralized and fair network, this approach helps to enhance the overall health and robustness of the system.

Resolution: The fix for this issue is still under development at the time of this audit. Client explanation of the ongoing fix and functionalities:

> We acknowledge and appreciate the concerns raised regarding potential centralization risks and perceived unfairness in the reward distribution model. However, the current reward split of 45% to validators and 55% to the PoV account is a strategic design choice that aligns with our goal of fostering a diverse, sustainable, and robust ecosystem.
>
> The PoV account is not a conventional account but a protocol-managed fund specifically designed to incentivize and reward contributors critical to the ecosystem's growth. This includes computational power(GPU) for training data, data quality, and other resource inputs, which are essential

for maintaining and scaling our ML Model. The rewards from this account are distributed through the PoV protocol, which is under active development.

We are implementing the PoV protocol using an ink! smart contract to ensure a flexible, transparent, and upgradable system for reward distribution. The PoV protocol is being designed to manage the registration of contributors, evaluate their contributions, and distribute rewards proportionally based on the value they provide to the network. Once it's functional and tested, it will be audited for security and robustness.

The PoV protocol is designed to mitigate centralization risks by dynamically distributing rewards based on actual contributions, ensuring that no single entity can dominate the system. This approach fosters a diverse and balanced network of participants, reinforcing decentralization within the ecosystem.

F-2024-4290 - Pool Liquidity Loss due to Insecure Pools Management - High

Description:

The `AMM` contract aims to support multiple token pairs. In such systems it is crucial to ensure the fund pools are separated and the funds are always taken from the appropriate pool.

The `removeLiquidity` function of the `AMM` contract counts the amount of token to withdraw using the formula `amount = shares * pair.token.balanceOf(address(this)) / pair.totalSupply`. The formula is based on the `AMM` contract balance which might include several distinct pools of the same token (in case the token is included to several trade pairs). The formula does not take the registered reserves into account.

```
function removeLiquidity(pair, shares) ... {
uint256 bal0 = pair.token0.balanceOf(address(this));
uint256 bal1 = pair.token1.balanceOf(address(this));

amount0 = (shares * bal0) / pair.totalSupply;
amount1 = (shares * bal1) / pair.totalSupply;
...
pair.token0.safeTransfer(msg.sender, amount0);
pair.token1.safeTransfer(msg.sender, amount1);
...
}
```

This may lead to unexpected withdraw of locked liquidity.

The `addLiquidity`, `removeLiquidity`, and `swap` functions of the `AMM` contract update the pair reserves with the current contract balance. The contract balance might include several distinct liquidity pools of the same token (in case the token is included to several trade pairs).

```
function addLiquidity(pair, amount0, amount1) ... {
...
uint256 balance0After = pair.token0.balanceOf(address(this));
uint256 balance1After = pair.token1.balanceOf(address(this));
...
_update(_pairId, balance0After, balance1After);
...
}

function removeLiquidity(pair, shares) ... {
...
uint256 bal0 = pair.token0.balanceOf(address(this));
uint256 bal1 = pair.token1.balanceOf(address(this));
...
_update(_pairId, bal0 - amount0, bal1 - amount1);
...
}

function swap(pair, tokenIn, amountIn) ... {
...
_update(
pair,
pair.token0.balanceOf(address(this)),
pair.token1.balanceOf(address(this))
);
```

```
...
}
```

This messes the token reserves, breaks the price calculations, allows liquidity providers to withdraw someone else's funds. Additionally, this introduces the pools drain ability using a flashloan.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Classification

Impact: 4/5

Likelihood: 3/5

Severity: High

Recommendations

Remediation: Ensure that fund pools for each token pair operate independently to prevent cross-contamination of funds. This can be achieved by creating distinct contract instances for each pair or aligning each pair's reserves with the actual amount of funds dedicated to the pair.

Modify the `addLiquidity`, `removeLiquidity`, and `swap` functions to accurately track and update the reserves based on the registered values rather than the overall contract balance.

Resolution: The Finding is fixed.

The pools are isolated and the contract does not update any pool reserves using the balances.

Evidences

Proof of Concept

Reproduce:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.22;

import "forge-std/Test.sol";
import "../src/AMM.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MockERC20 is ERC20 {
constructor(string memory name, string memory symbol) ERC20(name, s
ymbol) {}
```

```
function mint(address to, uint256 amount) public {
_mint(to, amount);
}
}

contract AMMTest is Test {
AMM public CONTRACT;
MockERC20 public T0;
MockERC20 public T1;
MockERC20 public T2;
uint256 public ID0;
uint256 public ID1;

address constant internal ATTACKER = address(0x12);
address constant internal LIQUIDITY_PROVIDER = address(0x13);
address constant internal USER = address(0x14);

function setUp() public {
T0 = new MockERC20("Token0", "TK0");
T1 = new MockERC20("Token1", "TK1");
T2 = new MockERC20("Token1", "TK1");
CONTRACT = new AMM();
ID0 = CONTRACT.createPair(address(T0), address(T1));
ID1 = CONTRACT.createPair(address(T0), address(T2));
}

function testBrokenSync() public {
uint256 initialLiquidity0 = 1_000 * 10**18;
uint256 initialLiquidity1 = 100_000 * 10**18;
uint256 attackerLiquidity = 100_000 * 10**18;

vm.startPrank(LIQUIDITY_PROVIDER);
T0.mint(LIQUIDITY_PROVIDER, initialLiquidity0);
T1.mint(LIQUIDITY_PROVIDER, initialLiquidity0);
T0.mint(LIQUIDITY_PROVIDER, initialLiquidity1);
T2.mint(LIQUIDITY_PROVIDER, initialLiquidity1);

T0.approve(address(CONTRACT), type(uint256).max);
T1.approve(address(CONTRACT), type(uint256).max);
T2.approve(address(CONTRACT), type(uint256).max);

CONTRACT.addLiquidity(ID0, initialLiquidity0, initialLiquidity0);
CONTRACT.addLiquidity(ID1, initialLiquidity1, initialLiquidity1);
vm.stopPrank();

vm.startPrank(ATTACKER);
T0.mint(ATTACKER, attackerLiquidity);
T1.mint(ATTAC
```

See more

Results:

```
[FAIL. Reason: assertion failed: 1990099009900990099010 < 102990099
009900990099010] testBrokenSync()
```

F-2024-4295 - Deposited Liquidity Misallocation due to Lack of Cashback - High

Description:

The `addLiquidity` function of the `AMM` contract allows users to specify the amounts of tokens to deposit. The function calculates the amounts of shares for each token deposited and mints the least one.

```
function addLiquidity(pair, amount0, amount1) ... {
...
shares = _min(
(amount0 * pair.totalSupply) / pair.reserve0,
(amount1 * pair.totalSupply) / pair.reserve1
);
...
require(shares > 0, "Shares = 0");
_mint(_pairId, msg.sender, shares);
...
}
```

Due to fluctuations in pair reserves (caused by sandwich attacks, for example), the calculated share amounts may vary significantly. The contract does not implement a cashback mechanism for any excessive tokens provided. Thus, the deposited funds are added directly to the pair reserves and are allocated among the liquidity providers.

This may lead to a significant portion of deposited funds being misallocated.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Classification

Impact: 4/5

Likelihood: 3/5

Severity: High

Recommendations

Remediation: Implement a cashback mechanism for the excessively deposited token to ensure that users receive the appropriate amount of shares for their deposits and any excess tokens are returned back.

Resolution: The Finding was fixed in the latest version of the codebase.

Evidences

Proof of Concept

Reproduce:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.22;

import "forge-std/Test.sol";
import "../src/AMM.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MockERC20 is ERC20 {
constructor(string memory name, string memory symbol) ERC20(name, s
ymbol) {}

function mint(address to, uint256 amount) public {
_mint(to, amount);
}
}

contract AMMTest is Test {
AMM public CONTRACT;
MockERC20 public T0;
MockERC20 public T1;
uint256 public ID;

address constant internal ATTACKER = address(0x12);
address constant internal LIQUIDITY_PROVIDER = address(0x13);
address constant internal USER = address(0x14);

function setUp() public {
T0 = new MockERC20("Token0", "TK0");
T1 = new MockERC20("Token1", "TK1");
CONTRACT = new AMM();
ID = CONTRACT.createPair(address(T0), address(T1));
}

function testLiquiditySandwichAttack() public {
uint256 initialLiquidity = 10_000 * 10**18;
uint256 attackerLiquidity = 100_000 * 10**18;
uint256 attackerSwap = 1_000_000 * 10**18;
uint256 userLiquidity = 10_000 * 10**18;

vm.startPrank(LIQUIDITY_PROVIDER);
T0.mint(LIQUIDITY_PROVIDER, initialLiquidity);
T1.mint(LIQUIDITY_PROVIDER, initialLiquidity);

T0.approve(address(CONTRACT), initialLiquidity);
T1.approve(address(CONTRACT), initialLiquidity);

CONTRACT.addLiquidity(ID, initialLiquidity, initialLiquidity);
vm.stopPrank();

vm.startPrank(ATTACKER);
T0.mint(ATTACKER, attackerSwap);
T0.mint(ATTACKER, attackerLiquidity);
T1.mint(ATTACKER, attackerLiquidity);

T0.approve(address(CONTRACT), type(uint256).max);
T1.approve(address(CONTRACT), type(uint256).max);
vm.stopPrank();

vm.startPrank(USER);
T0.mint(USER, userLiquidity);
T1.mint(USER, userLiquidity);

T0.approve(address(CONTRACT
```

See more

Results:

```
[FAIL. Reason: assertion failed: 1748534215916732642324 < 100000000
00000000000000] testLiquiditySandwichAttack()
```

F-2024-4297 - Locked Tokens Are Transferred To Arbitrary Address - High

Description: Whenever `lockTokens` is called it transfers an amount of tokens from user to the `AGCBridge` instance, however, then the `_burnTokens` is called to transfer this amount to the `burnAddress` address.

```
function lockTokens(uint256 amount, string memory substrateAddress)
external nonReentrant {
require(amount > 0, "Amount must be greater than 0");
require(agcToken.allowance(msg.sender, address(this)) >= amount, "T
oken allowance too low");

agcToken.safeTransferFrom(msg.sender, address(this), amount);
substrateAddresses[msg.sender] = substrateAddress;

emit TokensLocked(msg.sender, amount, substrateAddress);

_burnTokens(amount);
}

function _burnTokens(uint256 amount) internal {
require(agcToken.balanceOf(address(this)) >= amount, "Not enough to
kens in contract to burn");
agcToken.safeTransfer(burnAddress, amount);
emit TokensBurned(amount);
}
```

The `burnAddress` is an arbitrary address set by the protocol owner. Additionally, a validation prevents setting this state variable to zero address.

```
constructor(IERC20 _agcToken, address initialOwner, address _burnAd
dress) Ownable(initialOwner) {
require(_burnAddress != address(0), "Burn address cannot be zero ad
dress");
[...]
}

function updateBurnAddress(address newBurnAddress) external onlyOwn
er {
require(newBurnAddress != address(0), "Burn address cannot be zero
address");
burnAddress = newBurnAddress;
emit BurnAddressUpdated(newBurnAddress);
}
```

Such implementation pose a risk that tokens which were supposedly to be locked within the bridge contract are withdrawn and reused elsewhere. Additionally, this makes inconsistencies in the `AGC` token supply as additional amount of token can be minted on the Substrate chain, without reducing the supply on EVM chain.

Status: Accepted

Classification

Impact: 4/5

Likelihood: 4/5

Severity: High

Recommendations

Remediation: It is recommended to burn tokens be either transferring them to `address(0)` or by means of ERC20's `burn` function. Alternatively, it is recommended to remove burning of locked tokens at all.

Resolution: Within the received commit Id `2821bec684724513569ab00eb573db0f77b53006` no fix for the issue was identified.

Evidences

Proof of Concept

Reproduce:

1. As an owner, deploy the and configure the `AGCBridge` solution.
2. As an user, approve an amount of AGC tokens for the `AGCBridge` instance and call the `lockTokens` function.
3. Observe the tokens transfer. Note that `_burnTokens` function is called and an amount of tokens is transferred to the `burnAddress`.

F-2024-4303 - Vulnerable Dependencies and Outdated Substrate Version - High

Description: Several vulnerabilities were uncovered in project dependencies using `cargo audit`, posing significant risks to the codebase's security and stability. Below are the critical findings:

Timing variability in curve25519-dalek's Scalar29sub/Scalar52sub

- Crate: curve25519-dalek (Versions 2.1.3, 3.2.0, 4.0.0-rc.1)
- Severity: Low to Medium
- Date: 2024-06-18
- Solution: Upgrade to >=4.1.3

Double Public Key Signing Function Oracle Attack on ed25519-dalek

- Crate: ed25519-dalek (Version 1.0.1)
- Date: 2022-06-11
- Solution: Upgrade to >=2

Degradation of service in h2 servers with CONTINUATION Flood

- Crate: h2 (Version 0.3.20)
- Date: 2024-04-03
- Solution: Upgrade to ^0.3.26 OR >=0.4.4

Resource exhaustion vulnerability in h2 may lead to Denial of Service (DoS)

- Crate: h2 (Version 0.3.20)
- Date: 2024-01-17
- Solution: Upgrade to ^0.3.24 OR >=0.4.2

Tokens for named pipes may be delivered after deregistration

- Crate: mio (Version 0.8.8)
- Date: 2024-03-04
- Solution: Upgrade to >=0.8.11

Denial of service in Quinn servers

- Crate: quinn-proto (Version 0.9.4)
- Severity: 7.5 (high)
- Date: 2023-09-21
- Solution: Upgrade to ^0.9.5 OR >=0.10.5

rustls::ConnectionCommon::complete_io could fall into an infinite loop based on network input

- Crate: rustls (Versions 0.19.1, 0.20.8, 0.21.6)
- Severity: 7.5 (high)
- Date: 2024-04-19
- Solution: Upgrade to >=0.23.5 OR >=0.22.4, <0.23.0 OR >=0.21.11, <0.22.0

Multiple issues involving quote API

- Crate: shlex (Version 1.1.0)
- Date: 2024-01-21
- Solution: Upgrade to >=1.3.0

Unauthenticated Nonce Increment in snow

- Crate: snow (Version 0.9.2)
- Date: 2024-01-23
- Solution: Upgrade to >=0.9.5

Potential segfault in the time crate

- Crate: time (Version 0.1.45)
- Severity: 6.2 (medium)
- Date: 2020-11-18
- Solution: Upgrade to >=0.2.23

CPU denial of service in certificate path building

- Crate: webpki (Versions 0.21.4, 0.22.0)
- Severity: 7.5 (high)
- Date: 2023-08-22
- Solution: Upgrade to >=0.22.2

Additionally, 6 warnings related to unmaintained or unsound dependencies were discovered.

These issues primarily stem from using an outdated and archived version of Substrate. Specifically, the pallets are imported from the older Substrate repository, while a significantly newer version of these pallets exists in the Polkadot SDK. This situation not only introduces the aforementioned issues but also retains any unresolved Substrate-specific issues that may have been addressed in newer versions.

Assets:

- Substrate FRAME pallets [https://github.com/devolved-ai/argochain]

Status: Pending Fix

Classification

Impact: 4/5

Likelihood: 3/5

Severity: High

Recommendations

Remediation: To address these vulnerabilities and ensure the security and stability of your codebase, consider the following actions:

Review and Upgrade Dependencies: Although direct updates to most dependencies may not be feasible, consider upgrading to the latest version of the Polkadot SDK, specifically version v1.14.0 at the time of the audit. This upgrade will mitigate the most critical vulnerabilities present in the dependencies.

Regular Security Scans: Implement regular security checks using tools such as cargo audit to remain vigilant against potential vulnerabilities in your project's dependencies.

These actions will help fortify your application against known vulnerabilities and ensure ongoing security maintenance.

F-2024-4836 - Reserves to Balances Inconsistency due to Incorrect Fee Withdrawal Mechanism - High

Description:

The Automated Market Maker (AMM) contract exhibits a vulnerability in its fee management system. Specifically, the `swap` function adds the fee amount to a single state variable (`accumulatedFees`) regardless of the token involved in the swap. This is problematic because the `withdrawFees` function is hardcoded to withdraw fees in the form of `pairInfo[1].token0`, which can lead to unintended consequences.

- **Swap Function:** The `swap` function accumulates the fee amount in a single variable `accumulatedFees` without distinguishing between different tokens.

```
function swap(uint256 _pairId, ...) ... {
...
uint256 fee = (_amountIn * swapFee) / 10000;
accumulatedFees += fee;
...
}
```

- **Withdraw Fees Function:** The `withdrawFees` function transfers the entire accumulated fee amount from `pairInfo[1].token0`, regardless of which tokens were actually swapped or which tokens accumulated the fees.

```
function withdrawFees() onlyOwner ... {
...
IERC20(pairInfo[1].token0).safeTransfer(owner(), fees);
}
```

This approach can lead to an inconsistency between the contract's actual token balances and its recorded reserves. If a swap transaction involving a different token (not `pairInfo[1].token0`) accumulates fees, the withdrawal of fees in the form of `pairInfo[1].token0` could deplete its balance without reducing the corresponding reserves. As a result:

- **Liquidity Providers May Be Affected:** Liquidity providers may find themselves unable to withdraw their deposits due to depleted contract balances, even though the reserves indicate sufficient liquidity.
- **Swap Operations May Fail:** Subsequent swap operations could be blocked due to insufficient token balances, leading to disrupted functionality of the AMM.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Classification

Impact: 3/5

Likelihood: 4/5

Severity: High

Recommendations

Remediation:

- **Separate Fee Accumulation:** Implement a mapping structure to track accumulated fees separately for each token involved in the swaps. This can be done as follows:

```
mapping(address => uint256) public accumulatedFees;
```

Within the `swap` function, update the fee accumulation logic to:

```
accumulatedFees[address(tokenIn)] += fee;
```

- **Token-Specific Fee Withdrawal:** Modify the `withdrawFees` function to allow the owner to withdraw fees for each token independently:

```
function withdrawFees(address _token) onlyOwner ... {
uint256 fees = accumulatedFees[_token];
require(fees > 0, "No fees to withdraw");
accumulatedFees[_token] = 0;
IERC20(_token).safeTransfer(owner(), fees);
}
```

By adopting this approach, you ensure that fees are accurately accounted for and withdrawn in the correct token, maintaining consistency between the contract's token balances and reserves, and preventing disruptions in liquidity provision and swapping activities.

Resolution: The Finding is fixed.

The fees are accumulated and withdrawn at the per token basis.

Evidences

Proof of Concept

Reproduce:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.22;

import "forge-std/Test.sol";
import "../src/AMM_.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MockERC20 is ERC20 {
constructor(string memory name, string memory symbol) ERC20(name, s
ymbol) {}

function mint(address to, uint256 amount) public {
_mint(to, amount);
}
}

contract AMMTest is Test {
AMM public CONTRACT;
MockERC20 public T0;
MockERC20 public T1;
MockERC20 public T2;
uint256 public ID0;
uint256 public ID1;

address constant internal LIQUIDITY_PROVIDER = address(0x13);
address constant internal USER = address(0x14);

function setUp() public {
T0 = new MockERC20("Token0", "TK0");
T1 = new MockERC20("Token1", "TK1");
T2 = new MockERC20("Token1", "TK2");

T0.mint(LIQUIDITY_PROVIDER, 1);
T1.mint(LIQUIDITY_PROVIDER, 1);
T2.mint(LIQUIDITY_PROVIDER, 1);

CONTRACT = new AMM(address(1));
ID0 = CONTRACT.createPair(address(T0), address(T1));
ID1 = CONTRACT.createPair(address(T1), address(T2));
}

function testBrokenFees() public {
uint256 initialLiquidity = 100_000 * 10**18;
uint256 swapSize = 50_000 * 10**18;

vm.startPrank(LIQUIDITY_PROVIDER);
T0.mint(LIQUIDITY_PROVIDER, initialLiquidity);
T1.mint(LIQUIDITY_PROVIDER, initialLiquidity);
T1.mint(LIQUIDITY_PROVIDER, initialLiquidity);
T2.mint(LIQUIDITY_PROVIDER, initialLiquidity);

T0.approve(address(CONTRACT), type(uint256).max);
T1.approve(address(CONTRACT), type(uint256).max);
T2.approve(address(CONTRACT), type(uint256).max);

CONTRACT.addLiquidity(ID0, initialLiquidity, initialLiquidity);
CONTRACT.addLiquidity(ID1, initialLiquidity, initialLiquidity);
vm.stopPrank();

vm.startPrank(USER);
T1.mint(USER, swapSize);
T1.approve(address(CONTR
```

See more

Results:

```
[FAIL. Reason: ERC20InsufficientBalance(CONTRACT, [9.985e22], [9.99
9e22])] testBrokenFees()
```

F-2024-4282 - Inadequate Calculation of Residual Reward Amount - Medium

Description:

The issue stems from implementation of `era_payout`, which provides a logic for the reward distribution, happening each epoch.

Argochain/runtime/src/lib.rs:1196

```
impl pallet_staking::EraPayout<Balance> for CustomEraPayout {
fn era_payout(_total_staked: Balance, _total_issuance: Balance, _er
a_duration: u64) -> (Balance, Balance) {
let total_reward = FIXED_REWARD;

/* Calculations of rewards */

(final_validator_reward, total_reward)
}
}
```

The second balance returned by the function is intended to represent the amount of minted tokens remaining after the distribution of rewards to the validators. However, the current implementation erroneously sets this returned amount to a constant value representing the total reward for the era, rather than accurately calculating the residual amount after rewards have been allocated.

This error can distort the tokenomics of the system, leading to inaccurate reporting of the total token supply. Such discrepancies can undermine stakeholder trust and destabilize the perceived value and reliability of the token. Ensuring accurate calculation of residual rewards is essential for maintaining a balanced and trustworthy token economy.

Assets:

- Substrate FRAME pallets [https://github.com/devolved-ai/argochain]

Status: Fixed

Classification

Impact: 2/5

Likelihood: 5/5

Severity: Medium

Recommendations

Remediation: To address the issue in the `era_payout` implementation where the second balance returned incorrectly represents the total reward instead of the residual amount, the logic should be adjusted to accurately calculate the remaining tokens after distributing rewards to validators. The updated implementation should ensure that the second returned balance reflects the actual amount of tokens left after the validator rewards are deducted from the total reward.

F-2024-4151 - Lack Of Two-Step Ownership Transfer Pattern - Low

Description: The `AGCBridge` implement single step ownership transfer by means of `Ownable` Accidental transfer of ownership to unverified and incorrect address may result in loss of ownership. In such a case, access to every function protected by the ownership check will be permanently lost.

Assets:

- bridge-contract.sol [https://github.com/Devolved-AI/Devolved-AI_Argo-Wallet/tree/main/Bridge-backend]

Status: Accepted

Classification

Impact: 4/5

Likelihood: 1/5

Severity: Low

Recommendations

Remediation: It is recommended to implement a two-step ownership transfer pattern within the solution, such as OpenZepplin's `Ownable2Step` or `Ownable2StepUpgradeable`.

Resolution: The Client's Team accepted the finding with following comment:

> *During the deployment it has been ensured and the owner is set to the Argocoin Deployer which is the same account that deployed the Argocoin and standard practices are maintained to keep.*

F-2024-4153 - Return Value Of transferFrom Not Checked - Low

Description:

Within the `lockTokens` function the `transferFrom` is made on `IERC20` compatible `agcToken`. However, the boolean variable returned by this function is not being checked. Depending on the `agcToken` implementation, in rare cases, the `transferFrom` function may return `false` and silently fail, without the revert of the whole `lockTokens` operation.

```
agcToken.transferFrom(msg.sender, address(this), amount);
```

Thus, as a result, the function may perform accounting of locked tokens, without performing actual transfer.

Assets:

- bridge-contract.sol [https://github.com/Devolved-AI/Devolved-AI_Argo-Wallet/tree/main/Bridge-backend]

Status: Fixed

Classification

Impact: 3/5

Likelihood: 1/5

Severity: Low

Recommendations

Remediation:

It is recommended to check the return value of the `transferFrom()` function. This function returns a boolean value indicating the success or failure of the transfer operation. Alternatively, the SafeERC20 library of OpenZeppelin can be used to mitigate this issue and support any ERC20 implementation.

Resolution:

The protocol now uses the `safeTransferFrom()` function from the OpenZepplin's SafeERC20 library (commit Id: `0010cbf4e22883b48cf296f43c4c543b403b1913`).

F-2024-4298 - Lack Of Two-Step Ownership Transfer Pattern - Low

Description: The `AMM` contract currently follows the single-step ownership transfer model.

This allows accidental ownership transfers to arbitrary invalid addresses, resulting in permanent ownership loss.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Classification

Impact: 4/5

Likelihood: 1/5

Severity: Low

Recommendations

Remediation: Consider changing the ownership management model to a two-step process by replacing the `Ownable` dependency with `Ownable2Step`.

Resolution: The Finding is fixed.

The `Ownable2Step` contract is implemented and inherited by the `AMM` contract.

Observation Details

F-2024-4152 - Contract Has No Authorised Functions - Info

Description: The `AGCBridge` contract implements `Ownable`, however, it does not have any function marked with `onlyOwner` modifier or similar check. Thus, either the authorization is redundant, or it is not applied for desired functions.

```
contract AGCBridge is Ownable, ReentrancyGuard {
```

Assets:

- bridge-contract.sol [https://github.com/Devolved-AI/Devolved-AI_Argo-Wallet/tree/main/Bridge-backend]

Status: Fixed

Recommendations

Remediation: It is recommended to remove redundant functionality. Alternatively, it is recommended to review the implementation and apply authorisation whenever it is required.

Resolution: The protocol now uses `Ownable` to toggle on and off the pausable functionality (commit Id: `2821bec684724513569ab00eb573db0f77b53006`).

F-2024-4275 - Revert Messages Optimization - Info

Description:

Using revert string messages increases the size of a smart contract and its deployment costs.

Since Solidity version `0.8.4` custom errors are available.
Custom Errors provide a way for developers to define descriptive and semantically meaningful error conditions without relying on string messages.

The revert strings are used in the `getBalance`, `swap`, `removeLiquidity`, `addLiquidity`, `getPairInfo`, `createPair` functions and the `lock` modifier of the `AMM` contract.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Accepted

Recommendations

Remediation:

To reduce gas costs during contract deployment, consider using custom errors instead of revert strings.

Since Solidity version `0.8.26`, the IR compiler pipeline supports the use of Custom Errors within require statements. For the legacy pipeline or older Solidity versions, the pattern `if (!condition) revert CustomError()` should be used.

Resolution:

The Finding is fixed partially.

Some functions still follow the `require(condition, "revert message")`, however, most occurrences are eliminated.

F-2024-4276 - Floating Pragma - Info

Description: The project uses floating pragma `^0.8.22`.

This may result in the contracts being deployed using the wrong pragma version, which is different from the one they were tested with. For example, they might be deployed using an outdated pragma version which may include bugs that affect the system negatively.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Recommendations

Remediation: Consider locking the pragma version and avoid using a floating pragma in the final deployment. Consider known bugs for the compiler version that is chosen.

Resolution: The Finding is fixed.

The language version is pinned to `0.8.25`.

F-2024-4293 - Lack of Token Standard Compliance - Info

Description:

Standardizing liquidity pool shares as ERC-20 or ERC-1155 tokens is a common practice in decentralized finance (DeFi) applications. This standardization enables users to transfer shares between accounts, manage shares in their wallets, and interact seamlessly with other DeFi protocols.

The `AMM` contract does not adhere to any recognized token standards such as ERC-20 or ERC-1155. The absence of compliance with these standards reduces user flexibility and interoperability, limiting the usability and appeal of the `AMM` contract.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Accepted

Recommendations

Remediation:

Implement the ERC-1155 standard within the contract to manage liquidity pool shares. ERC-1155 is particularly suitable for handling multiple token types within the same contract, which is advantageous for an AMM supporting various token pairs.

Consider inheriting the ERC-1155 token implementation and utilizing the `_mint`, `_burn`, and `balanceOf` functions provided by the ERC-1155 standard.

Resolution:

The Finding is not fixed.

The ERC-1155 compliance is not implemented.

F-2024-4294 - Public Functions That Should Be External - Info

Description: Functions intended to be invoked exclusively from external sources should be designated as `external` rather than `public`.

This is essential to enhance both gas efficiency and the overall contract readability.

The visibility of the `getBalance` function can be restricted from `public` to `external`.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Recommendations

Remediation: Consider updating functions which are exclusively utilized by external entities from their current `public` visibility to `external` visibility.

Resolution: The Finding is fixed.

The contract does not implement any `public` functions anymore.

F-2024-4296 - Reentrancy Guard Duplication - Info

Description: The `lock` modifier in the `AMM` contract duplicates the functionality of the `nonReentrant` modifier from the `ReentrancyGuard` contract. Both modifiers are applied to the `addLiquidity`, `removeLiquidity`, and `swap` functions, resulting in unnecessary Gas expenses.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Recommendations

Remediation: Consider removing the `lock` modifier and `getUnlocked` function. Instead utilize the `nonReentrant` modifier to safeguard the functions against reentrancy.

Resolution: The Finding is fixed.

The custom reentrancy guard implementation is removed. The redundant `unlocked` state variable is still presented in the implementation.

F-2024-4304 - Presence of Unused Template Pallet - Info

Description:

The codebase contains a `pallets/template` folder, which was copied from the standard Substrate pallet template. This folder is unnecessary and is not included in the workspace.

The presence of this unused pallet clusters the codebase and can be misleading for developers. It may cause confusion regarding its purpose and relevance, potentially leading to wasted time and effort in understanding or modifying it. Moreover, it detracts from the overall cleanliness and maintainability of the codebase.

Assets:

- Substrate FRAME pallets [https://github.com/devolved-ai/argochain]

Status: Fixed

Recommendations

Remediation:

To maintain a clean and efficient codebase, it is recommended to:

- **Remove the Template Pallet:** Delete the `pallets/template` folder from the repository to avoid any confusion and unnecessary clutter.
- **Update Documentation:** Ensure that any documentation or developer guidelines are updated to reflect the removal of this folder, clarifying that it was an unused template example not relevant to the current codebase.

Implementing these recommendations will enhance the clarity and maintainability of the code, making it easier for developers to navigate and work with the codebase.

F-2024-4818 - Unnecessary Access Restriction on a View Function - Info

Description: The `getAccumulatedFees` function of the `AMM` contract is a declared as view. The function is marked as `onlyOwner` what is unnecessary as the function does not perform any state updates that should be restricted.

The blockchain data is transparent and restricting access to view functions does not actually restrict users from retrieving the information.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Recommendations

Remediation: Consider removing the `onlyOwner` modifier to align with the intended purpose of view functions.

Resolution: The Finding is fixed.

The view functions are implemented without the unnecessary restrictions.

F-2024-4830 - Sign of Unfinalized Code: Unused Internal Functions - Info

Description:

The following internal functions of the `AMM` contract are never executed and, though, are redundant.

- `_transferTokensToContract`
- `_calculateAndMintShares`
- `_prepareSwapInfo`
- `_calculateAmountOut`
- `_updateReserves`

Along with the functions, there are the `AddLiquidityParams` and `SwapInfo` structures which are utilized only in the redundant functions.

Keeping redundant functions can lead to increased contract complexity, as well as occupying valuable bytecode space, which can result in higher deployment costs.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Recommendations

Remediation:

Review the necessity of these functions. Consider removing them to improve contract maintainability and deployment costs.

Resolution:

The Finding is fixed.

The redundant `internal` functions are eliminated.

F-2024-4831 - Lack of ERC-20 Operation Success Validation - Info

Description:

According to the ERC-20 standard, the `transferFrom` and `transfer` functions return `bool success`. The return value should be properly handled.

However, in the `wrap` and `unwrap` functions of the `AMM` contract the value is ignored (`IWETH(wethAddress).transfer`, `IWETH(wethAddress).transferFrom`).

This may lead to inconsistencies if the `wethAddress` does not fully match the ERC-20 standard.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Recommendations

Remediation:

Consider using `safeTransfer` and `safeTransferFrom` functions of the `SafeERC20` library to check whether the transfers were successful.

Resolution:

The Finding is fixed.

The `SafeERC20` library usage is implemented.

F-2024-4832 - Sign of Unfinalized Code: Unused Allowance Functionality - Info

Description:

The `AMM` contract implements the `increaseAllowance`, `decreaseAllowance`, `_approve` functions. The functions manage the `_allowances` state mapping.

However, the `_allowances` state mapping is never used in the contract except of the mentioned functions. This makes the function redundant or signs that a codebase part is missing.

Keeping redundant functions can lead to increased contract complexity, as well as occupying valuable bytecode space, which can result in higher deployment costs.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Accepted

Recommendations

Remediation:

Review the necessity of these functions. Consider removing them to improve contract maintainability and deployment costs.

Resolution:

The Finding is not fixed.

The functionality is not eliminated,

F-2024-4833 - Redundant View Getters for Public State Variables - Info

Description: The `AMM` contract defines the `getAccumulatedFees`, `getTotalSupply`, `getReserve0`, `getReserve1` functions which directly return associated public state variable value.

State variables declared as `public` automatically generate the external getter functions, though, the manual implemented getters are redundant.

Keeping redundant functions can lead to higher deployment costs.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Recommendations

Remediation: Consider removing the functions to lower the deployment costs.

Resolution: The Finding is fixed.

The redundant getters are eliminated.

F-2024-4834 - Block WETH Functionality due to Lock of Native Token Receiving - Info

Description:

The `unwrap` function of the `AMM` contract calls the `withdraw` function of `WETH` which is supposed to transfer native tokens to the caller. The native token transfer requires the `AMM` contract implement the `receive` function.

However, the `AMM` contract implements the `receive` function in the way of blocking of any incoming transfers.

```
receive() external payable {
revert("Contract does not accept ETH directly");
}
```

This makes the `unwrap` function always end with a revert.

The users are still able to unwrap native token using the `WETH` contract directly.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Fixed

Recommendations

Remediation:

Consider removing the native token wrap functionality along with the `receive` function from the contract. Alternatively, implement the `receive` function to validate the sender is `WETH` contract and accept the incoming funds.

Resolution:

The Finding is fixed.

The WETH wrapping functionality is eliminated. The redundant `wethAddress` state variable is still presented in the implementation.

F-2024-4835 - Sign of Unfinalized Code: Redundant Constants - Info

Description: The `AMM` contract defines immutable `weth` variable and `LP_FEE_SHARE` constant which are never used.

Redundant constants lead to confusions during development and potentially signing about unfinalized codebase.

Assets:

- AMM.sol [https://github.com/Devolved-AI/Swap]

Status: Accepted

Recommendations

Remediation: Consider removing the redundant declarations.

Resolution: The Finding is not fixed.

There are some redundant variables implemented in the `AMM` contract.

According to the Client notice, the `LP_FEE_SHARE` constant is user to determine which fee part is distributed across the Liquidity Providers, however, the functionality in not presented in the final codebase and all the fees are distributed to the contract owner.

Disclaimers

Hacken Disclaimer

The blockchain protocol given for audit has been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in the protocol source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other protocol statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of the blockchain protocol.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

Technical Disclaimer

Blockchain protocols are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the protocol can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited blockchain protocol.

Appendix 1. Severity Definitions

Severity	Description
Critical	Vulnerabilities that can lead to a complete breakdown of the blockchain network's security, privacy, integrity, or availability fall under this category. They can disrupt the consensus mechanism, enabling a malicious entity to take control of the majority of nodes or facilitate 51% attacks. In addition, issues that could lead to widespread crashing of nodes, leading to a complete breakdown or significant halt of the network, are also considered critical along with issues that can lead to a massive theft of assets. Immediate attention and mitigation are required.
High	High severity vulnerabilities are those that do not immediately risk the complete security or integrity of the network but can cause substantial harm. These are issues that could cause the crashing of several nodes, leading to temporary disruption of the network, or could manipulate the consensus mechanism to a certain extent, but not enough to execute a 51% attack. Partial breaches of privacy, unauthorized but limited access to sensitive information, and affecting the reliable execution of smart contracts also fall under this category.
Medium	Medium severity vulnerabilities could negatively affect the blockchain protocol but are usually not capable of causing catastrophic damage. These could include vulnerabilities that allow minor breaches of user privacy, can slow down transaction processing, or can lead to relatively small financial losses. It may be possible to exploit these vulnerabilities under specific circumstances, or they may require a high level of access to exploit effectively.
Low	Low severity vulnerabilities are minor flaws in the blockchain protocol that might not have a direct impact on security but could cause minor inefficiencies in transaction processing or slight delays in block propagation. They might include vulnerabilities that allow attackers to cause nuisance-level disruptions or are only exploitable under extremely rare and specific conditions. These vulnerabilities should be corrected but do not represent an immediate threat to the system.

Appendix 2. Scope

The scope of the project includes the following components from the provided repository:

Scope Details	
Repository	https://github.com/Devolved-AI/Argochain
	https://github.com/Devolved-AI/Devolved-AI_Argo-Wallet/blob/main/Bridge-backend/bridge-contract.sol
	AMM.sol Contract
Commit	19e66003f426e6aba0e5f5bf26afb578450dc4af
	d5893a7ad1b53f67c569fbd53483ed6064f35176
Requirements	N/A
Technical Requirements	N/A